JARVIS Project

Setup & Usage Guide

Project basis: Powerfull_Jarvis_using_OpenAI.py and apikey.py

1. Project Overview

This project is a Python-based voice assistant named Jarvis. As written in the supplied source, it combines speech recognition, text-to-speech, web browser commands, weather lookup, time, jokes, Wikipedia summaries, and a default OpenAI-powered response path.

The main program imports OpenAI, pyttsx3, SpeechRecognition, webbrowser, requests, datetime, wikipedia, and random.

The API key is imported from apikey.py and assigned to the OpenAI client configuration.

2. Recommended Project Folder

Keep the two supplied Python files in the same project directory:

```
JARVIS/
■■■ Powerfull_Jarvis_using_OpenAI.py
■■■ apikey.py
```

Important: The main script contains from apikey import api_data, so apikey.py needs to be importable from the same project directory (or otherwise be on Python's import path).

3. Python Environment

Create a virtual environment so the project's packages stay isolated from other Python projects.

```
python -m venv venv
```

Activate it on Windows:

```
venv\Scripts\activate
```

On macOS/Linux:

```
source venv/bin/activate
```

Then verify Python is available:

```
python --version
```

4. Install the Required Packages

The supplied main script imports these third-party packages:

```
openai
pyttsx3
SpeechRecognition
requests
wikipedia
```

```
PyAudio
```

A practical installation command is:

```
pip install openai pyttsx3 SpeechRecognition requests wikipedia PyAudio
```

The script also imports Python standard-library modules such as webbrowser, datetime, and random; these do not need to be installed separately.

If PyAudio installation fails on your Windows machine, the failure is usually related to audio/portaudio packaging rather than the Jarvis source itself. Install a compatible PyAudio build for your Python version, then retry.

5. Configure the OpenAI API Key

The supplied apikey.py defines the API key variable:

```
api_data = "Your API Key"
```

Replace the placeholder value with your API key. The main script imports that variable and assigns it to the OpenAI API configuration.

Security: Do not publish your real API key in GitHub, screenshots, videos, or shared source code. For a real project, consider using an environment variable instead of committing a key to a Python file.

6. Configure the Weather API

The weather function contains a separate placeholder:

```
api_key = "your_weather_api_key" # Replace with your weather API key
```

The supplied code constructs an OpenWeatherMap request and asks for metric units. Therefore, weather commands require a valid OpenWeatherMap API key in the location shown in the source.

7. Run Jarvis

From the project folder, with the virtual environment activated, run:

```
python Powerfull_Jarvis_using_OpenAI.py
```

On startup, the program calls the text-to-speech function with the greeting "Hello! How can I assist you today?" and then enters its continuous listening loop.

8. Voice Commands Available in the Supplied Code

The following commands are explicitly handled by the main loop:

Voice phrase / intent	What Jarvis does
open youtube	Opens YouTube in the web browser.
open google	Opens Google in the web browser.
open github	Opens GitHub in the web browser.
time	Reads the current system time.
weather	Asks for a city name, retrieves weather data, then speaks the result.

Voice phrase / intent	What Jarvis does
joke	Speaks a randomly selected joke from the built-in list.
wikipedia	Asks what to search for and speaks a two-sentence Wikipedia summary.
bye	Says goodbye and exits the loop.
anything else	Passes the query to the OpenAI Reply() function.

9. How the Voice Pipeline Works

The supplied code follows this basic flow:

```
Microphone → SpeechRecognition → text query → command matching
↓
built-in command OR OpenAI Reply()
↓
pyttsx3 speech output
```

The takeCommand() function records microphone audio and uses Google speech recognition with the en-in language setting. If recognition fails, it prints "Say That Again...." and returns "None".

10. OpenAI Response Path

For queries that do not match the built-in commands, the script calls Reply(). That function builds a prompt in the form:

```
Chando: {question}
Jarvis:
```

It then requests a completion and returns the generated text, which is printed and spoken aloud.

Compatibility note: The supplied source uses the legacy OpenAI completion pattern with openai.Completion(), completion.create(), and the text-davinci-002 engine. This guide documents the code as supplied rather than silently rewriting it. If the installed OpenAI Python package no longer supports this interface, the OpenAI section will need to be migrated to a currently supported API before that part of Jarvis can run.

11. Suggested First Test

After launching the program, test the components one at a time:

• Say "time" to test the time function and text-to-speech.

• Say "joke" to test the built-in joke function.

• Say "open google" to test browser control.

• Say "wikipedia" and provide a topic to test the Wikipedia path.

• Say "weather" and then a city to test the weather API.

• Finally, try a general question to test the OpenAI response path.

12. Troubleshooting

Problem	Check
ModuleNotFoundError	Activate the virtual environment and install the missing package with pip.

Problem	Check
Microphone not detected	Check that Windows has a working microphone, that the application has microphone permission, and that PyAudio is installed correctly.
Speech recognition repeatedly returns None	Check microphone input and internet access. The supplied code uses Google speech recognition and returns None when recognition raises an exception.
Weather does not work	Verify the OpenWeatherMap API key in get_weather() and check that the requested city can be resolved.
OpenAI section fails	Check the API key and note that the supplied code uses a legacy OpenAI completion interface/model. Package/API compatibility may require migration.
No voice output	Check Windows audio output and the pyttsx3/SAPI5 setup. The supplied code initializes pyttsx3 with the SAPI5 driver.

13. Quick Setup Checklist

- Create a project folder.
- Place Powerfull_Jarvis_using_OpenAI.py and apikey.py together.
- Create and activate a Python virtual environment.
- Install the imported third-party packages.
- Set the OpenAI API key in apikey.py.
- Set the OpenWeatherMap API key in get_weather().
- Connect/check a working microphone and speakers.
- Run python Powerfull_Jarvis_using_OpenAI.py.
- Test built-in commands before testing the OpenAI response path.

14. Source Notes

This guide was prepared from the two uploaded source files. The main program imports the API key from apikey.py, initializes the OpenAI completion object, defines speech input/output, weather/time/joke helpers, and implements the command loop.

The API key file itself contains a placeholder variable named api_data; no real secret was reproduced in this guide.

Project status: The guide describes the supplied implementation. It does not claim that every dependency or legacy API used by the source is compatible with the latest package versions.